Artificial Intelligence –
Deep learning 5.1: ***Sequence models***

Master in Artificial Intelligence & Computational Engineering
Luis Montesano

Universidad Zaragoza
1542

Today

- Sequential data and recurrent neural networks (RNNs)
- Training RNNs: backpropagation through time, vanishing and exploding gradients
- Gated units: LSTM and GRU
- Deep RNNs and applications: video classification, dense captioning
- Other sequence models: temporal convolutions (TCN, WaveNet)
- Encoder–decoder (seq2seq) and the bottleneck that motivates attention
- Attention in practice: image captioning
- Homework: a char-RNN on your own text

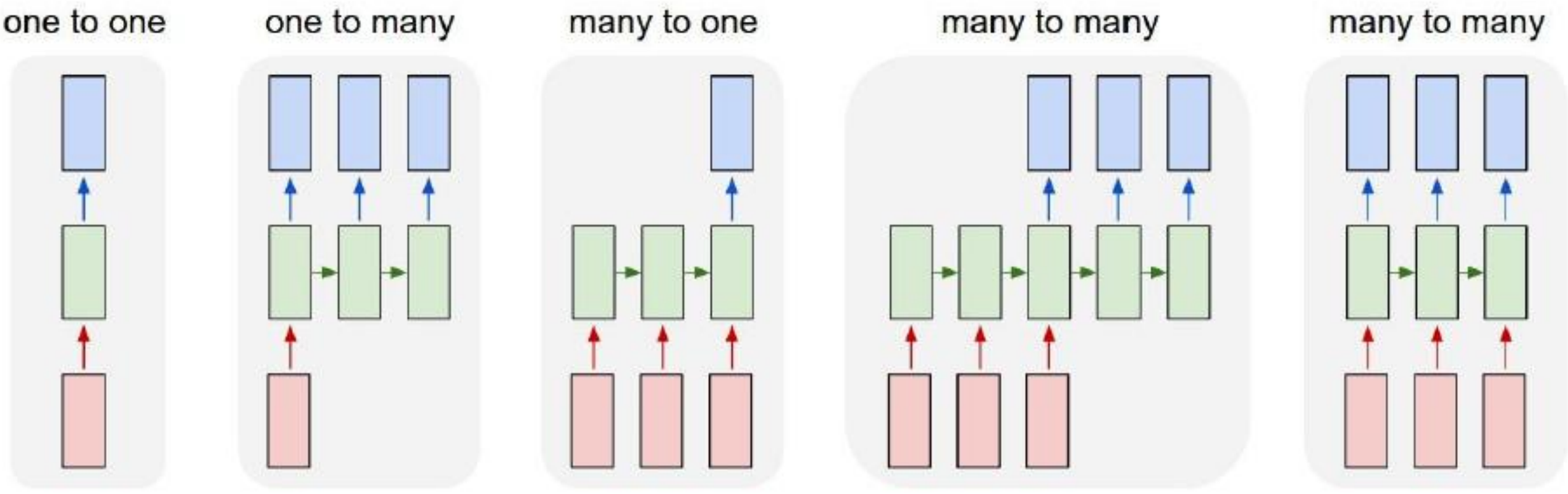
Sequential data: order matters

Sequential data: order matters

- **Text and speech**: words and sounds only make sense in order
- **Video**: frames over time
- **Sensor and financial data**: time series
- **Length varies** from one example to the next
  - Feed-forward nets and CNNs expect a fixed-size input

Recurrent networks for sequences

- Video, speech, text → **Recurrent NN**



Fei-Fei Li & Justin Johnson & Serena Yeung
http://cs231n.stanford.edu

  - can handle **non-fixed length input/output**

Unrolling an RNN: one set of shared weights

- repeat (as many input/output/steps as needed)
- **share the weights** across iterations

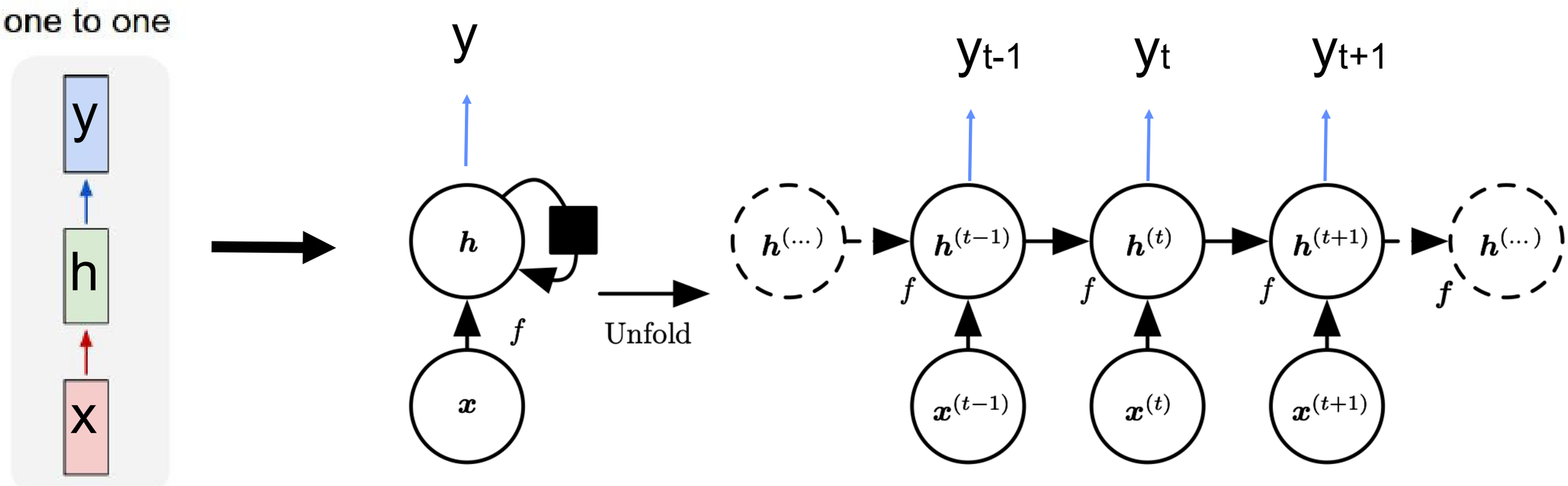


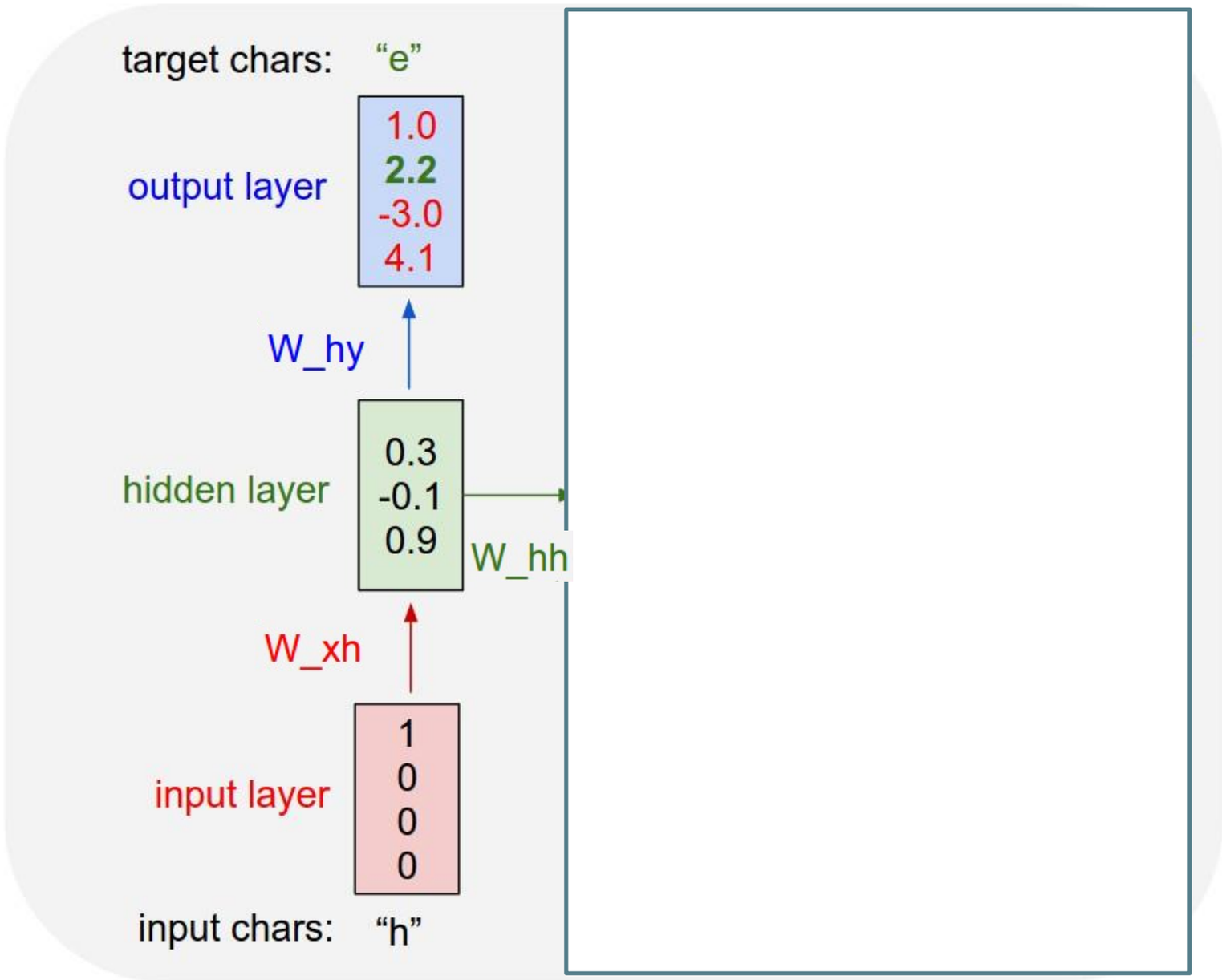
Figure 10.2 - Deep Learning book. Ian Goodfellow

Toy example: a character-level RNN

Main ingredients on a toy example:

- time: t
- hidden state: h
- “vocabulary”: {h,e,l,o}
- weights: $\mathbf{W_{xh}}$, $\mathbf{W_{hh}}$, $\mathbf{W_{hy}}$

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$
$$y_t = W_{hy}h_t$$



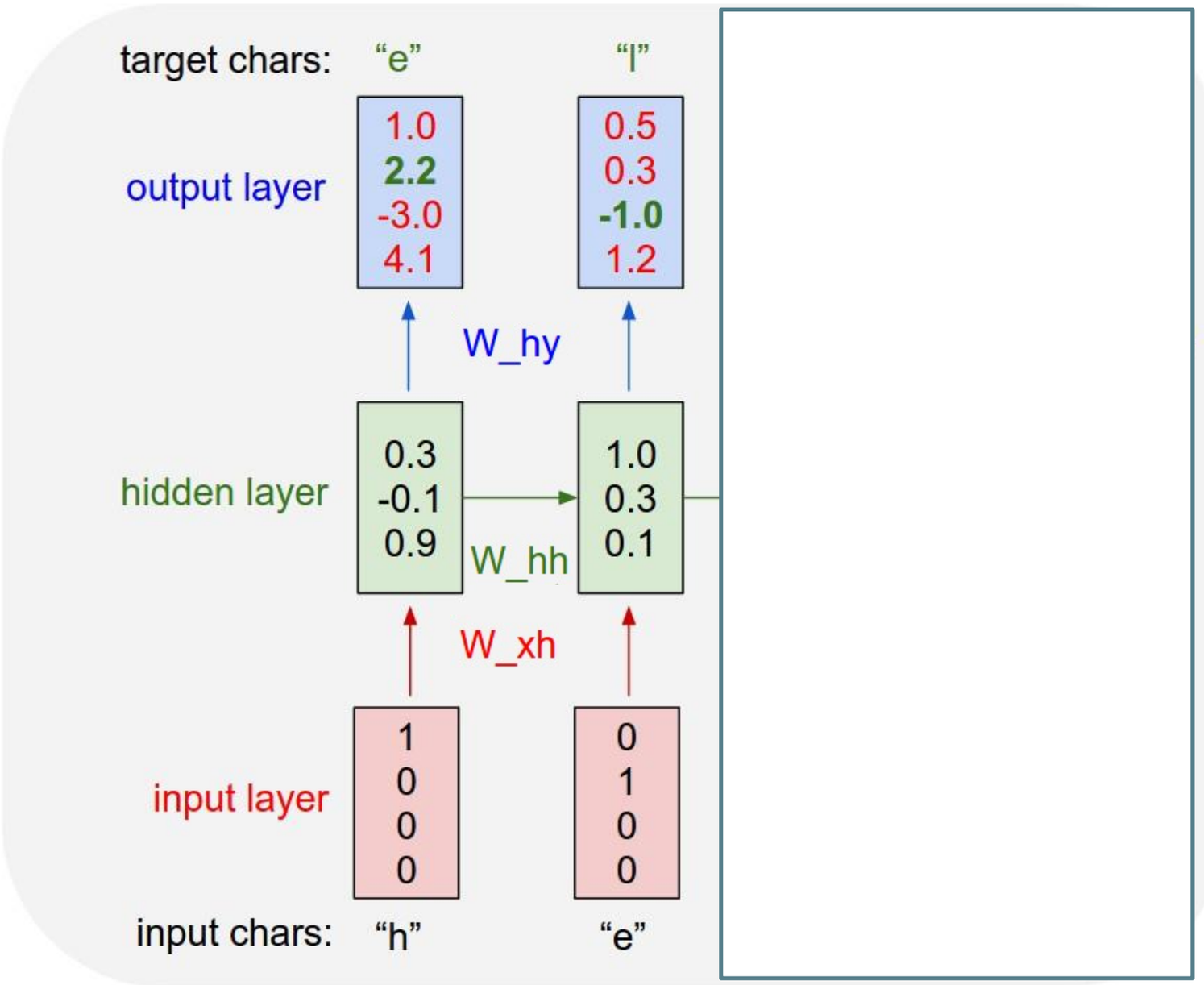
Andrej Karpathy
http://karpathy.github.io/2015/05/21/rnn-effectiveness/

Toy example: a character-level RNN

Main ingredients on a toy example:

- time: t
- hidden state: h
- “vocabulary”: {h,e,l,o}
- weights: $\mathbf{W_{xh}}$, $\mathbf{W_{hh}}$, $\mathbf{W_{hy}}$

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$
$$y_t = W_{hy}h_t$$



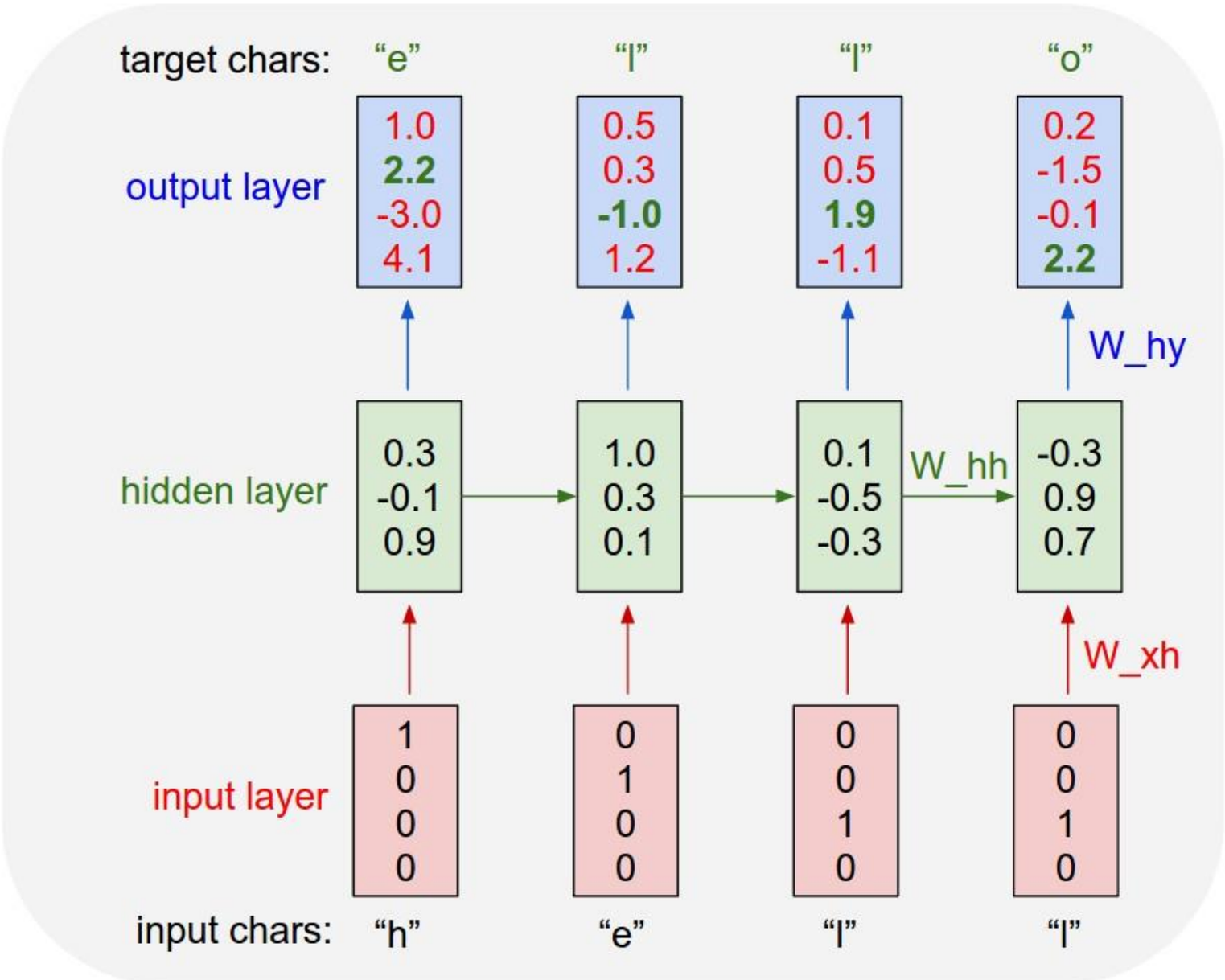
Andrej Karpathy
http://karpathy.github.io/2015/05/21/rnn-effectiveness/

Toy example: a character-level RNN

Main ingredients on a toy example:

- time: t
- hidden state: h
- “vocabulary”: {h,e,l,o}
- weights: $\mathbf{W_{xh}}$, $\mathbf{W_{hh}}$, $\mathbf{W_{hy}}$

$$h_t = \tanh(\boldsymbol{W}_{hh} h_{t-1} + \boldsymbol{W}_{xh} x_t)$$
$$y_t = \boldsymbol{W}_{hy} h_t$$



Andrej Karpathy
http://karpathy.github.io/2015/05/21/rnn-effectiveness/

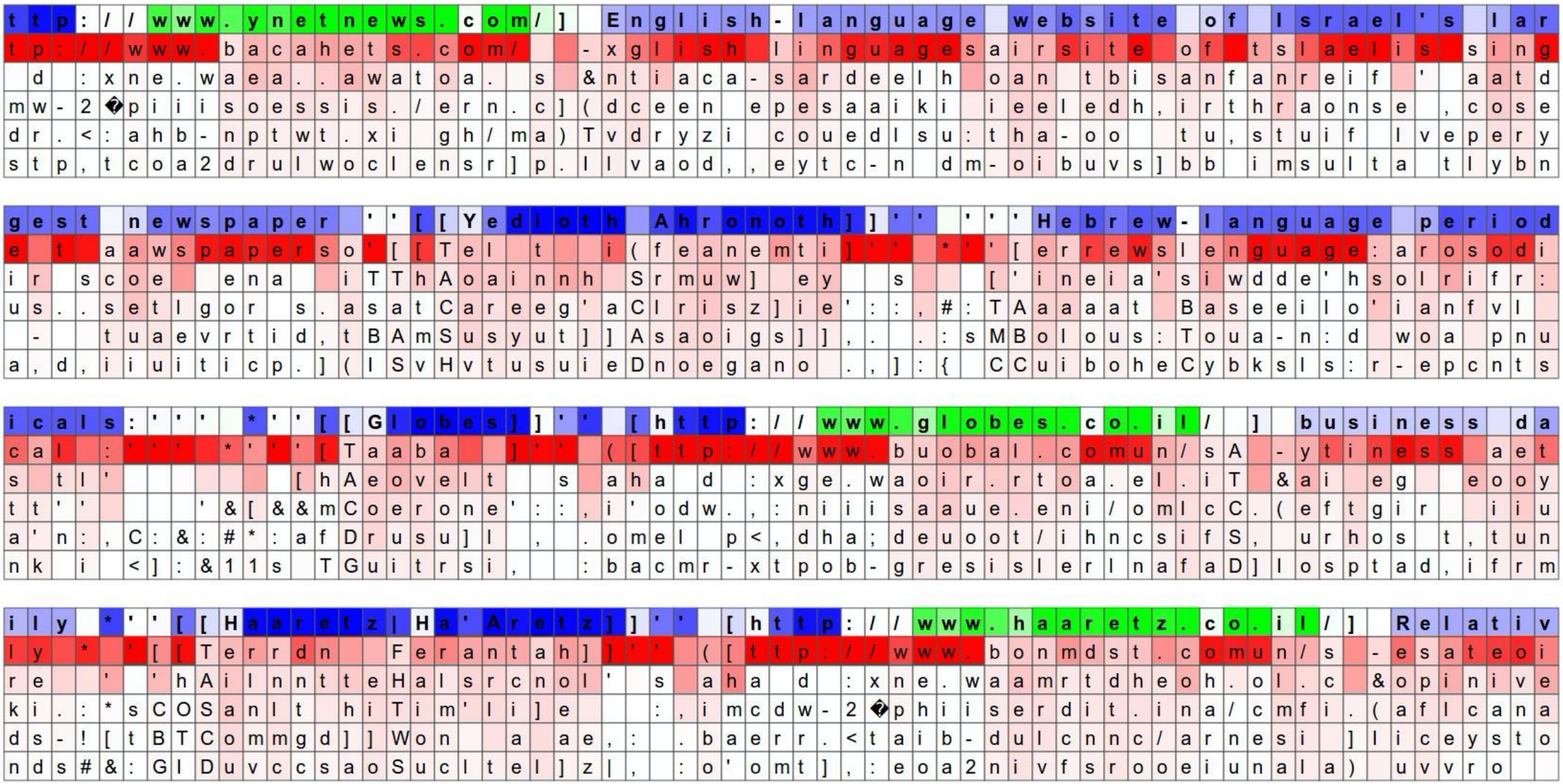
What a vanilla RNN can learn: char-RNN examples

Interesting examples from a ***vanilla*** version

- text generator (Shakespeare, Wikipedia page, …)
- code generator! (LaTeX, C, …)

http://karpathy.github.io/2015/05/21/rnn-effectiveness/

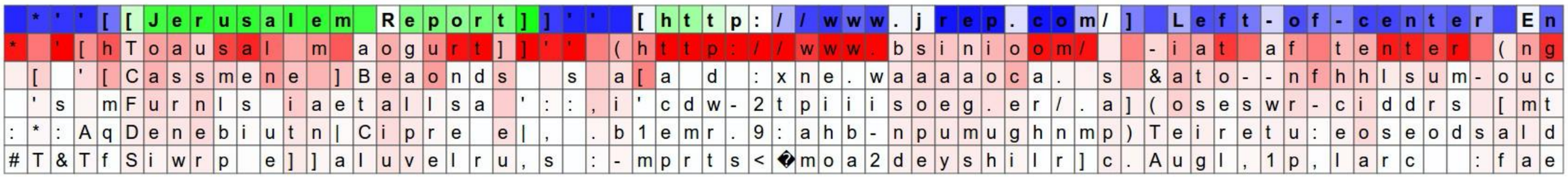
Interpretable neurons: one cell detects URLs

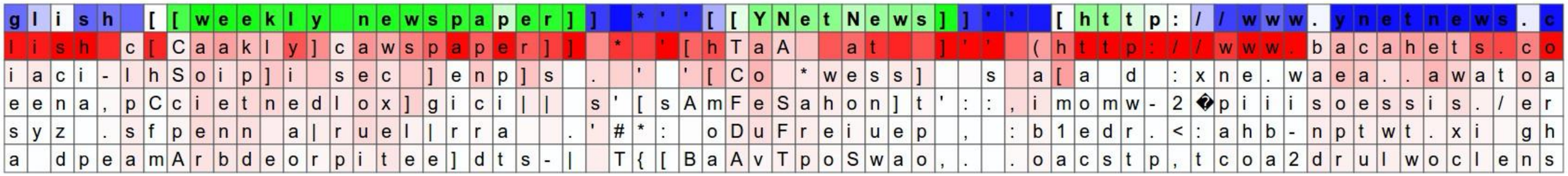


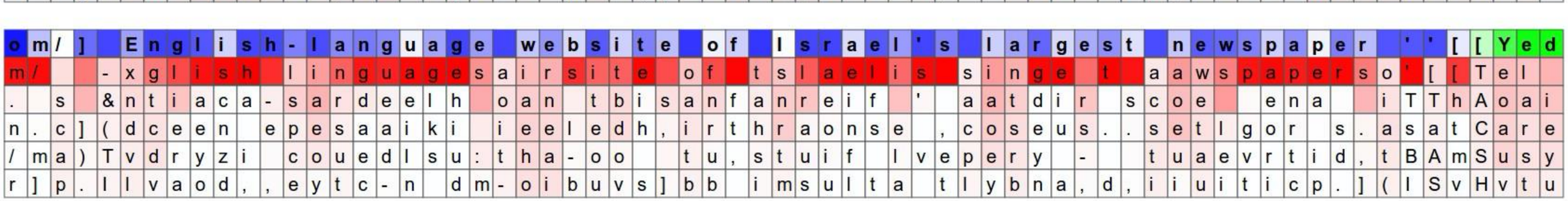
Top row of each block: input sequence; below: the 5 most likely next characters

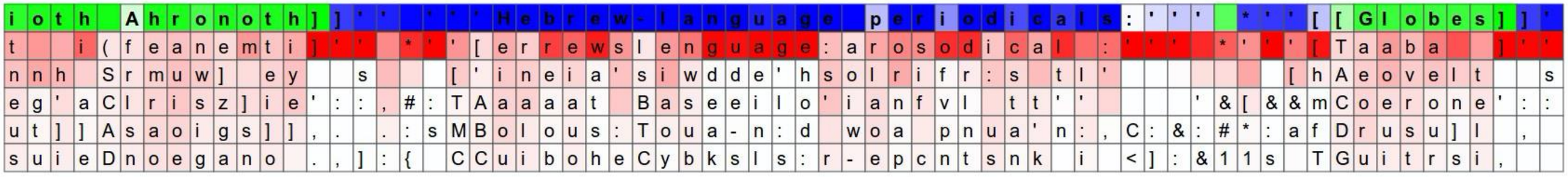
http://karpathy.github.io/2015/05/21/rnn-effectiveness/

Interpretable neurons: one cell tracks [[ ]] links









http://karpathy.github.io/2015/05/21/rnn-effectiveness/

Vanilla RNNs miss long-range dependencies

- Dependencies over many steps are hard to learn: gradients **vanish or explode** through time

- Next: why this happens (BPTT) and how gated cells fix it

Training RNNs: backpropagation through time

- **Unroll** the RNN over T steps → a deep feed-forward network whose layers **share** W_{xh}, W_{hh}, W_{hy}
- Total loss = sum of the per-step losses:

$$L = \sum_{t=1}^{T} L_t\left(y_t,\, y_t^*\right)$$

- Standard backprop (4.3) on the unrolled graph. Gradients of a shared weight **add up** over time:

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^{T}\sum_{k=1}^{t} \frac{\partial L_t}{\partial h_t} \underbrace{\left(\prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}}\right)}_{\text{how } h_k \text{ affects } h_t} \frac{\partial^+ h_k}{\partial W_{hh}}$$

  - ∂^+: immediate derivative, treating h_{k-1} as a constant
- **Truncated BPTT**: backpropagate through the last K steps only (memory, speed)

Goodfellow, Bengio, Courville. Deep Learning, ch. 10.2 · R. Pascanu, T. Mikolov, Y. Bengio. On the difficulty of training recurrent neural networks. ICML 2013

Why gradients vanish or explode

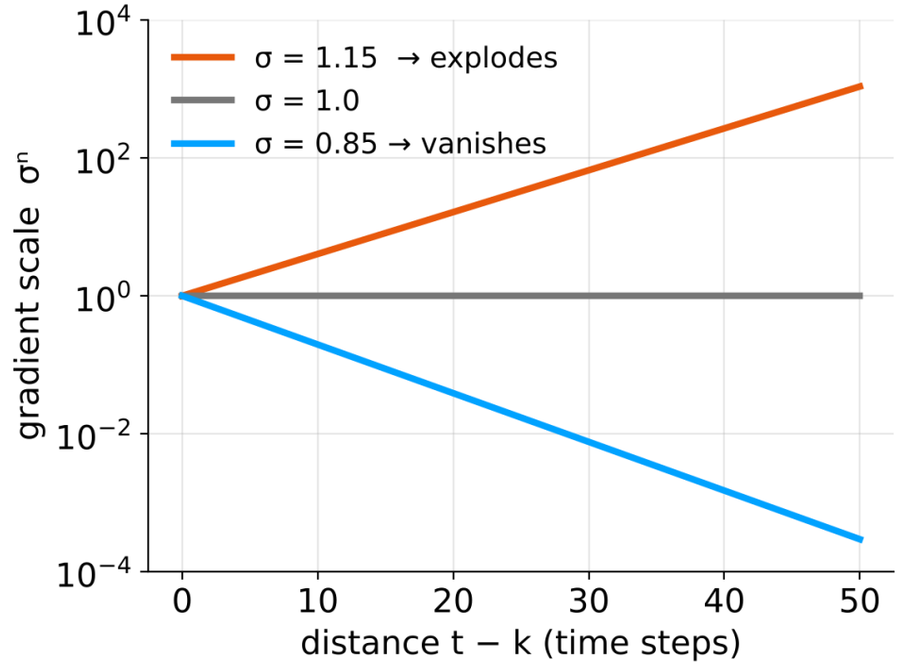
- Each factor of the product is a Jacobian:

$$\frac{\partial h_j}{\partial h_{j-1}} = \mathrm{diag}\left(1 - h_j^2\right) W_{hh}$$

- Since |tanh′| ≤ 1:

$$\left\| \prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} \right\| \leq \sigma_{\max}(W_{hh})^{\,t-k}$$

- $\sigma_{max} < 1 \rightarrow$ contributions from far-away steps shrink exponentially: **vanishing** gradient, long-range dependencies are not learned
- $\sigma_{max} > 1 \rightarrow$ they can grow exponentially: **exploding** gradient (loss spikes, NaN)



R. Pascanu, T. Mikolov, Y. Bengio. On the difficulty of training recurrent neural networks. ICML 2013 (σ_max = largest singular value)

Fixing the gradient problems

- **Exploding** → gradient clipping: rescale the gradient if its norm is too large

$$\text{if} \quad \|g\| > \tau : \qquad g \leftarrow \tau \frac{g}{\|g\|}$$

  - Keeps the direction, limits the step. One line in PyTorch: torch.nn.utils.clip_grad_norm_
- **Vanishing** → change the architecture
  - Gated units (LSTM, GRU): an **additive** path for information and gradients, like the skip connections of ResNets
  - Helpers: orthogonal initialisation of W_{hh}, layer normalisation
  - Truncated BPTT saves memory and time but does not fix vanishing: it cuts the long-range gradients
- Or remove the long chain of multiplications altogether
  - Temporal convolutions (TCN) and attention: short paths between any two time steps

R. Pascanu, T. Mikolov, Y. Bengio. On the difficulty of training recurrent neural networks. ICML 2013

LSTM: long short-term memory

- **LSTM networks** - Long short-term memory cells
  - The most widely used RNN cell
  - More sophisticated hidden function than the basic RNN
  - Capable of learning long-term dependencies

$$h_t = \tanh\left(W\begin{pmatrix}h_{t-1}\\x_t\end{pmatrix}\right) \longrightarrow \begin{pmatrix}i\\f\\o\\g\end{pmatrix} = \begin{pmatrix}\sigma\\\sigma\\\sigma\\\tanh\end{pmatrix} W\begin{pmatrix}h_{t-1}\\x_t\end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

S. Hochreiter, J. Schmidhuber. Long short-term memory. Neural Computation, 1997 · Forget gate: F. Gers, J. Schmidhuber, F. Cummins. Learning to forget. Neural Computation, 2000

LSTM: a cell state with gates

- **LSTM networks** - Long short-term memory cells
  - The most widely used RNN cell
  - More sophisticated hidden function than the basic RNN
  - Capable of learning long-term dependencies

$$h_t = \tanh\left(W\begin{pmatrix}h_{t-1}\\x_t\end{pmatrix}\right)$$

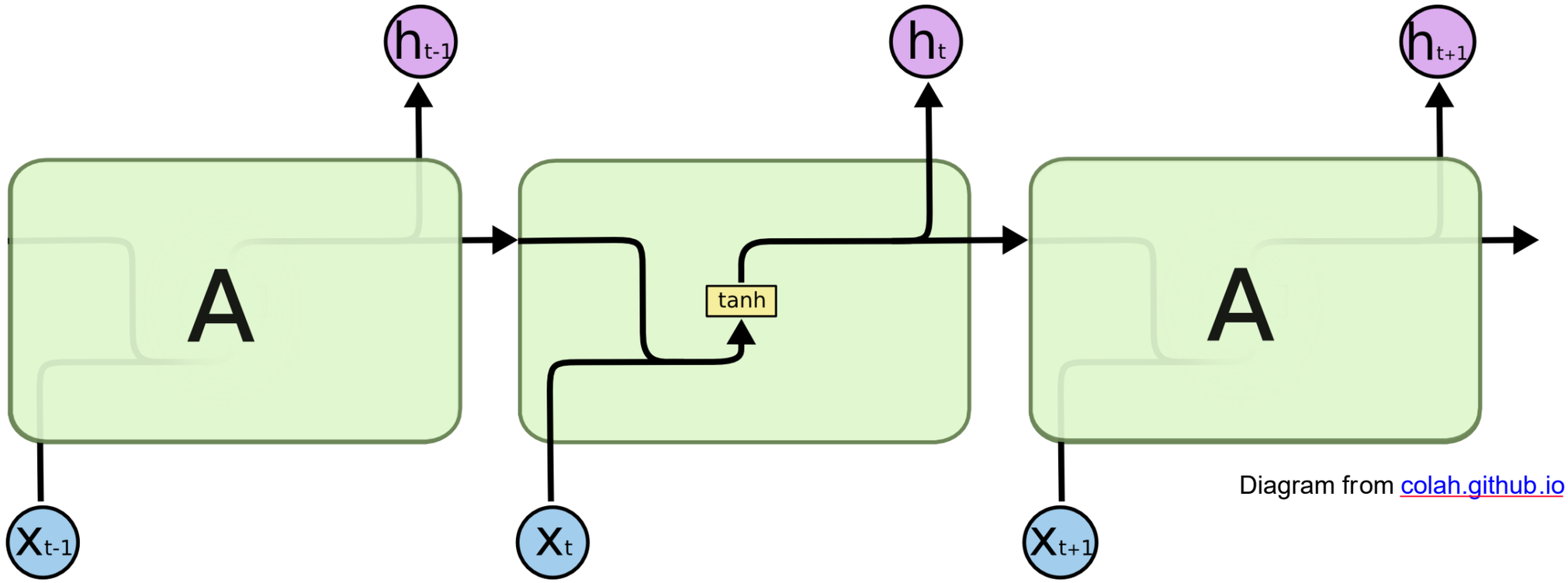


Diagram from colah.github.io

LSTM: inside the cell

- **LSTM networks** - Long short-term memory cells
  - The most widely used RNN cell
  - More sophisticated hidden function than the basic RNN
  - Capable of learning long-term dependencies

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

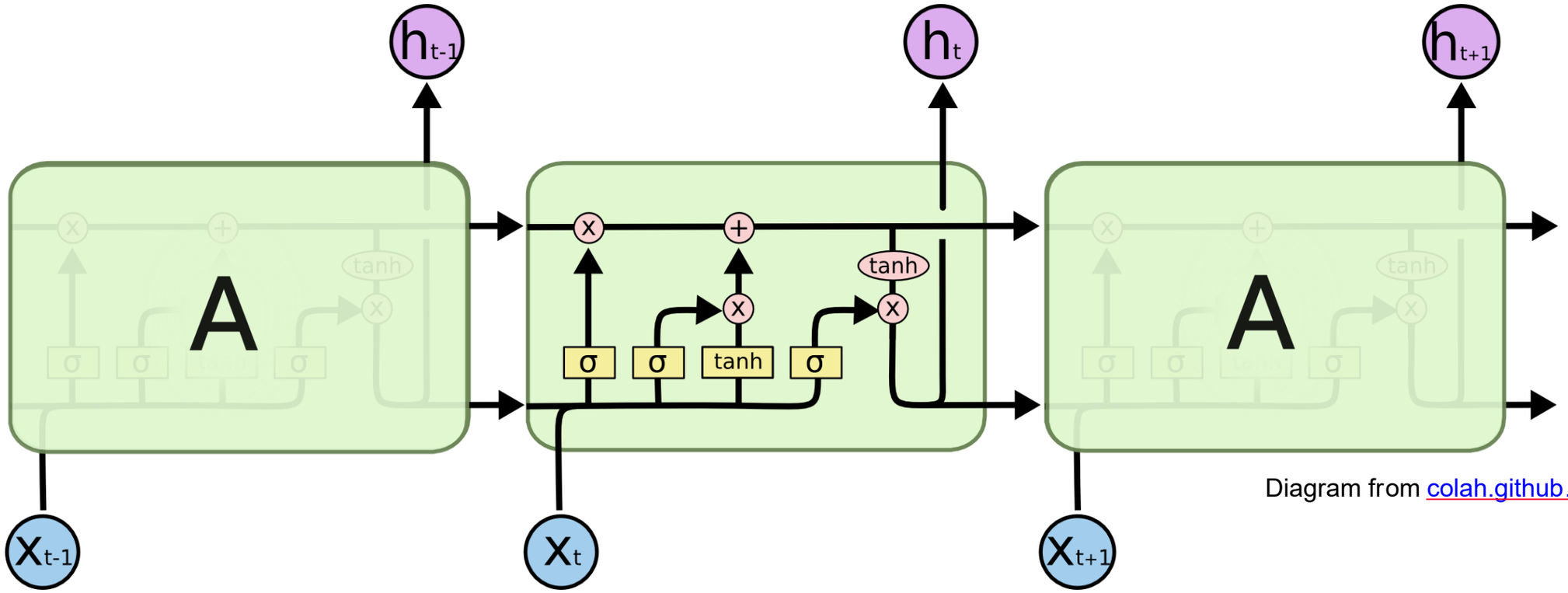
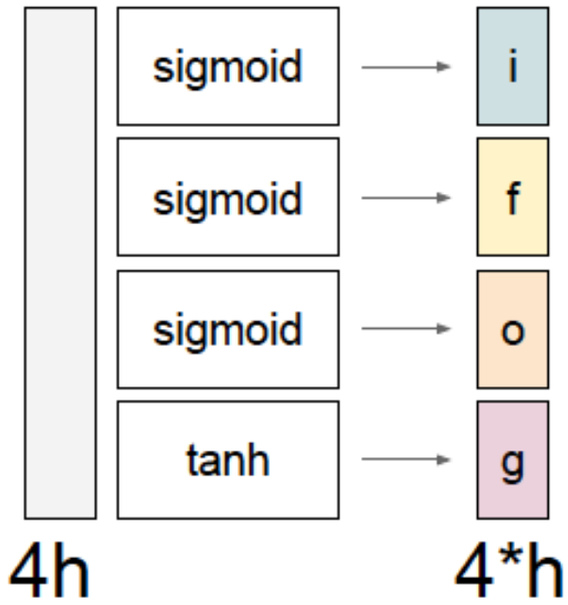


Diagram from colah.github.io

LSTM gates: forget, input, cell, output

- **LSTM networks** - Long short-term memory cells
  - **f: forget** gate.
    Erase cell or not?
  - **i: input** gate.
    Write cell or not?
  - **g: cell *gate*.**
    How much to write to cell
  - **o: output** gate.
    How much to reveal cell value

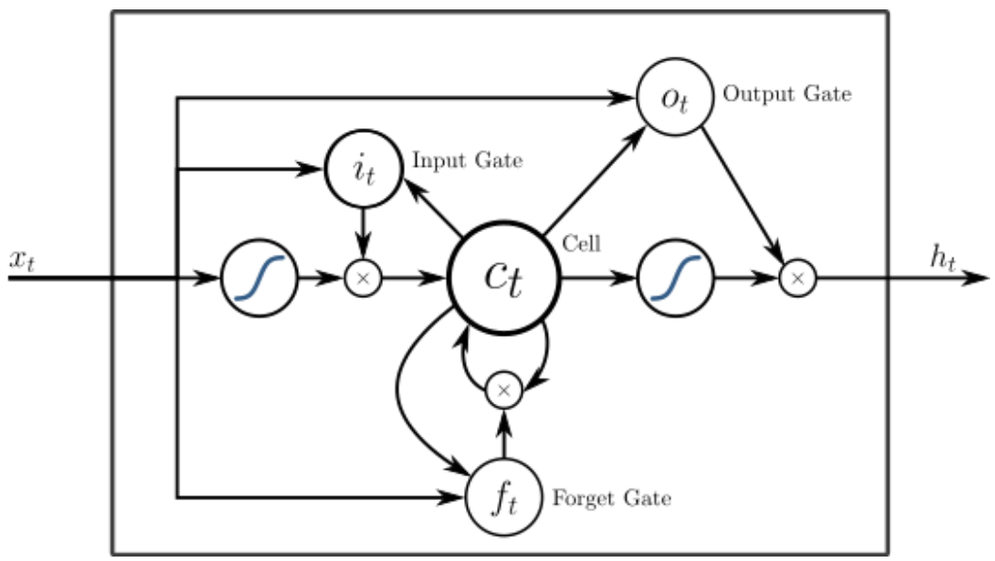


Fei-Fei Li & Justin Johnson & Serena Yeung
http://cs231n.stanford.edu

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$



Wikipedia

Why gates help, and the GRU

- The LSTM cell state is updated **additively**:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \quad \Longrightarrow \quad \frac{\partial c_t}{\partial c_{t-1}} = \mathrm{diag}(f_t) \;\; (\text{direct path})$$

  - Forget gate $f_t \approx 1 \rightarrow$ information and gradients cross many steps without being multiplied by W_{hh}

- **GRU** (Cho et al., 2014): lighter variant, no separate cell state; update gate z_t, reset gate r_t

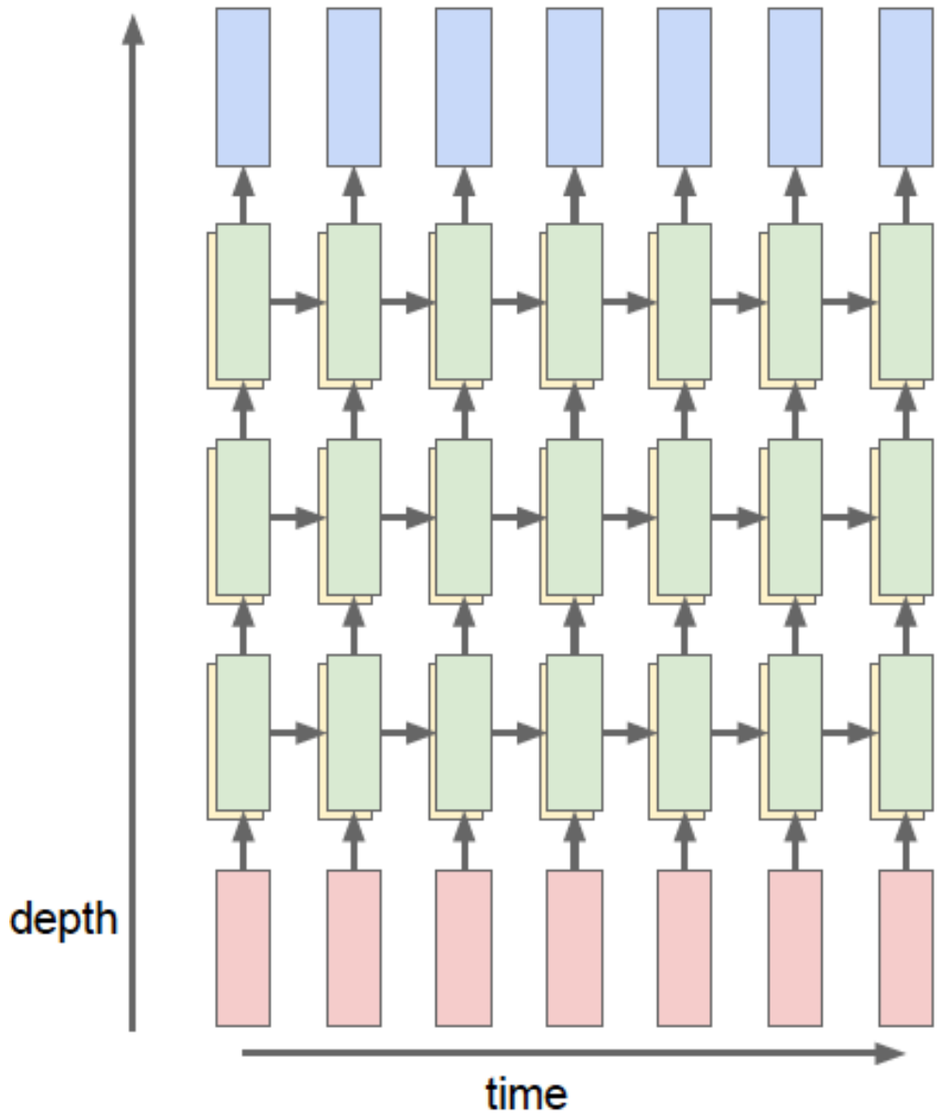
$$z_t = \sigma\big(W_z[h_{t-1}, x_t]\big) \qquad r_t = \sigma\big(W_r[h_{t-1}, x_t]\big)$$
$$\tilde{h}_t = \tanh\big(W[r_t \odot h_{t-1},\, x_t]\big) \qquad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

  - 3 weight blocks instead of 4 $\rightarrow$ fewer parameters; similar accuracy on many tasks (nn.LSTM, nn.GRU)

K. Cho et al. Learning phrase representations using RNN encoder–decoder for statistical machine translation. EMNLP 2014 · biases omitted · conventions differ: PyTorch nn.GRU swaps z and 1 − z

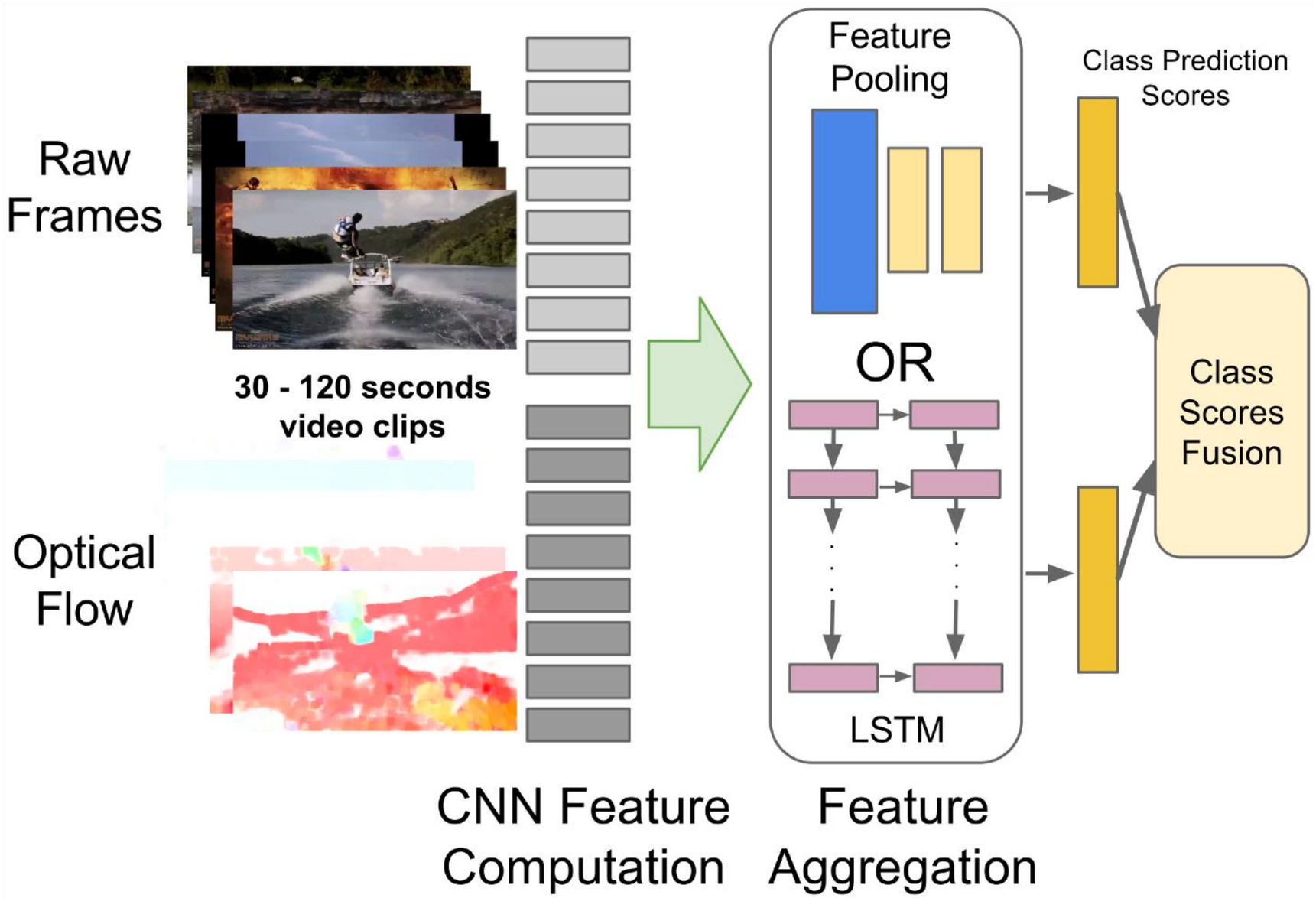
Stacking RNN layers, and other variants

- Many successful examples and variations:
  - Many variations of LSTM cells
  - Multi-layer RNN
  - Bidirectional RNN
    - reads both directions
    - needs the whole input first



RNN applications: action recognition in video

- **Action Classification**. Video and Flow (temporal data)

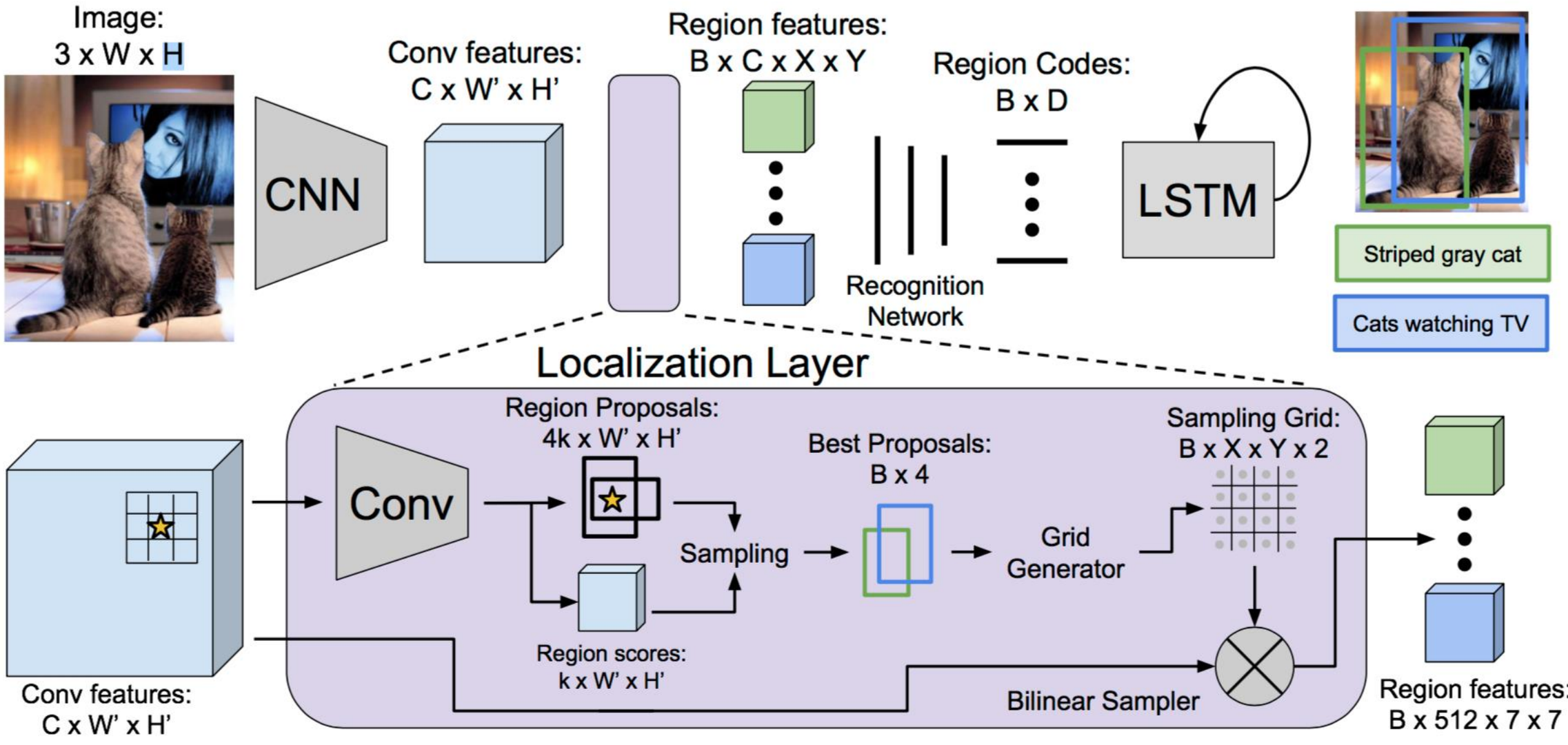


Results: [Beyond short snippets]

Beyond Short Snippets: Deep Networks for Video Classification.
Joe Yue-Hei Ng et al. Google. CVPR 2015.

RNN applications: dense captioning

- **Multimodal problems**. Image and Text. Dense Captions (other types of sequential data)



DenseCap: Fully Convolutional Localization Networks for Dense Captioning.
Johnson, Justin and Karpathy, Andrej and Fei-Fei, Li. CVPR 2016

[Results: DenseCap project page]

Other *temporal* processing: TCN

- TCN - **Temporal Convolutional Network**
  - Causal CNN / Convolutional Markov Model

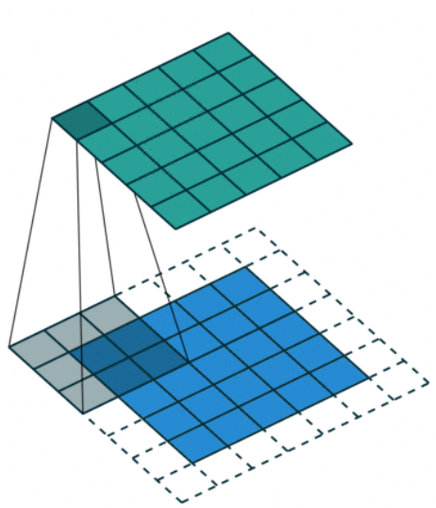
$$p(\mathbf{x}) = \prod_{t=1}^{T} p(x_t \mid x_1, \ldots, x_{t-1})$$

In practice each sample sees only the previous samples within the receptive field

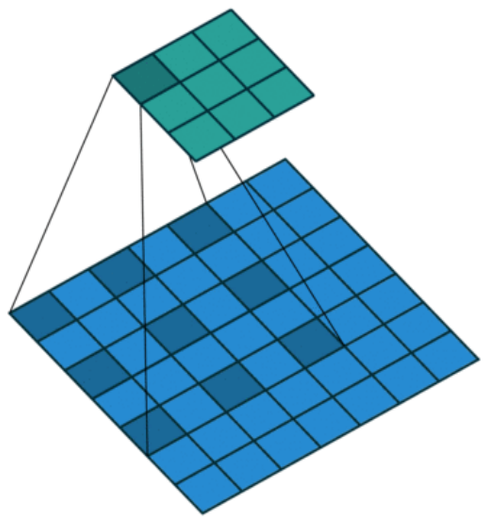
- Receptive field grows with depth and dilation (next slides)

Other *temporal* processing: TCN

- TCN - Temporal Convolutional Network
  - Causal CNN / Convolutional Markov Model
  - 1D dilated convolutions



standard convolution



dilated convolution

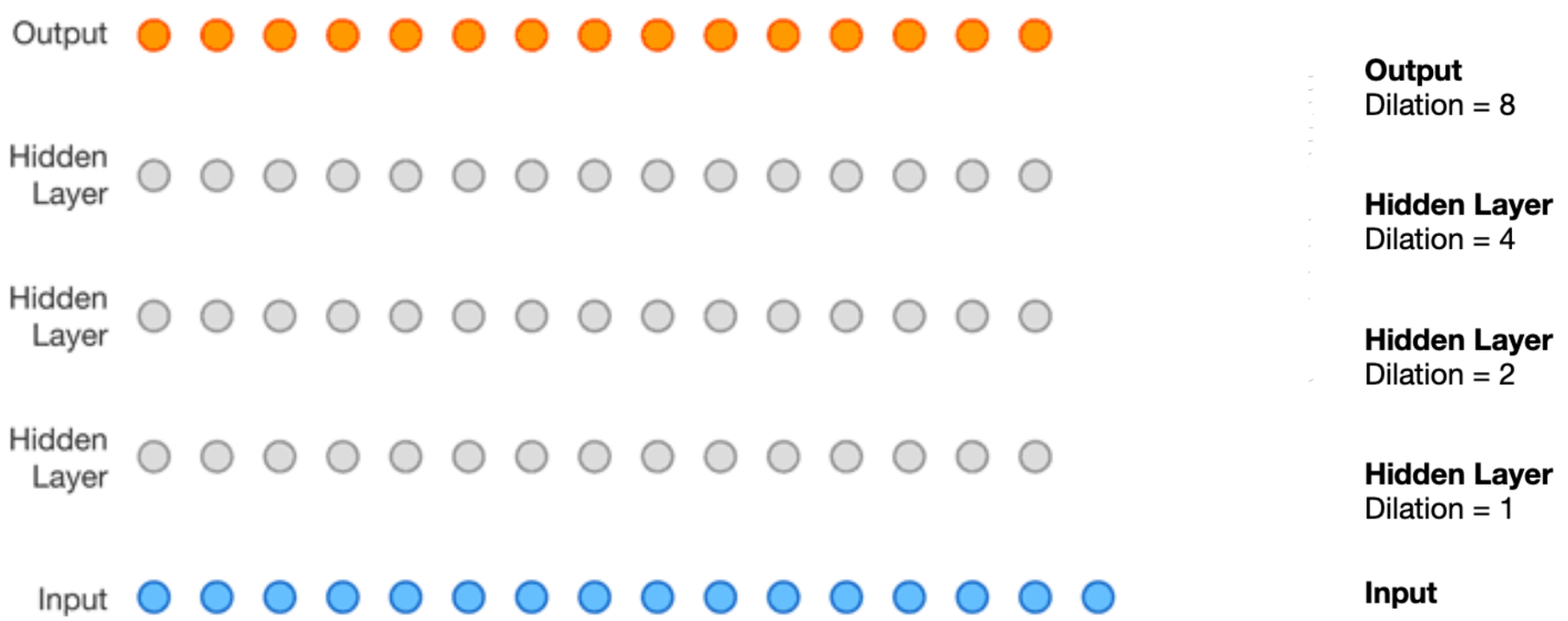
2D illustrations; a TCN applies the same idea in 1D and causally (only past inputs). Fig.: V. Dumoulin, F. Visin, conv_arithmetic

Other *temporal* processing: TCN

- TCN - Temporal Convolutional Network
  - Causal CNN / Convolutional Markov Model
  - 1D dilated convolutions

https://deepmind.com/blog/article/wavenet-generative-model-raw-audio

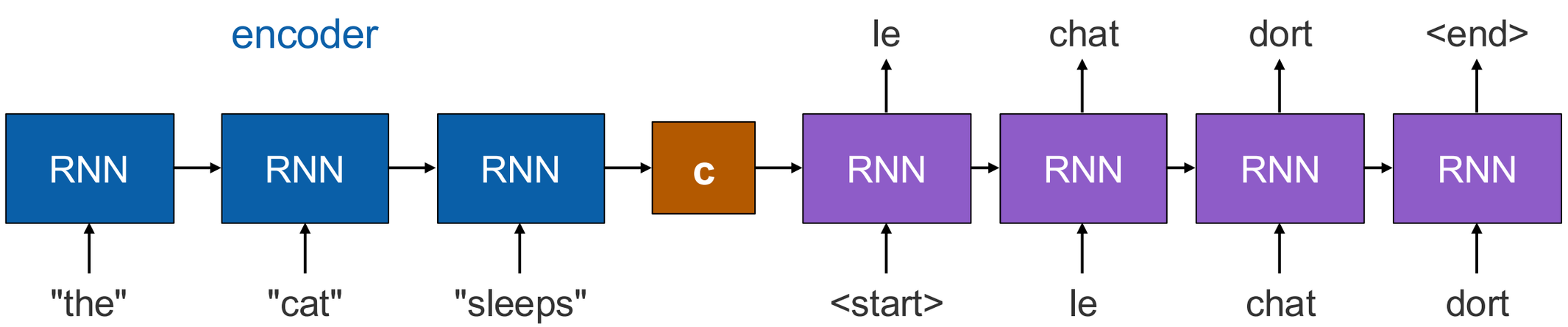
- **WaveNet**: stack of dilated causal 1D conv layers



Oord, Aaron van den, et al. "Wavenet: A generative model for raw audio." arXiv preprint arXiv:1609.03499 (2016).

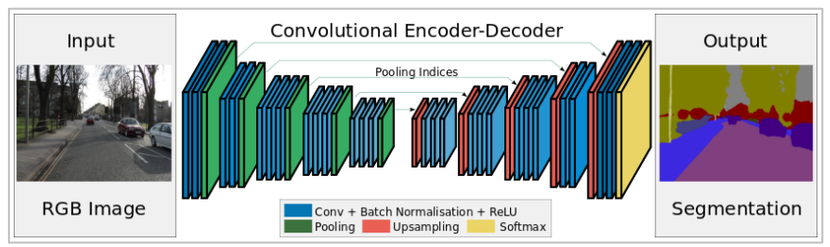
Seq2seq: encoder–decoder RNNs

- Same **encoder–decoder** as SegNet (4.4), over **time** instead of space; **lengths may differ**
- **Encoder** RNN reads $x_1 \ldots x_T$: last state = **context vector** $c = h_T$
- **Decoder** RNN generates $y_1, y_2, \ldots$ one token at a time until <end>



decoder: each output is fed back as the next input

$$p(y_1, \ldots, y_{T'} \mid x_1, \ldots, x_T) = \prod_{t=1}^{T'} p(y_t \mid y_{<t}, c)$$



Recall 4.4: SegNet, the same idea over space

I. Sutskever, O. Vinyals, Q. V. Le. Sequence to sequence learning with neural networks. NeurIPS 2014 · Fig.: SegNet, PAMI 2017

Training the decoder: teacher forcing

- **Training**: feed the ground-truth previous token y^*_{t-1} as decoder input, not the model's own guess
  - Every step gets the correct context → stable training, no sampling needed; steps run in parallel only in non-recurrent decoders (5.2)
- Loss = cross-entropy of the next token (maximum likelihood, as in 1.2):

$$\mathcal{L} = -\sum_{t=1}^{T'} \log p\left(y_t^* \mid y_{<t}^*,\, c\right)$$

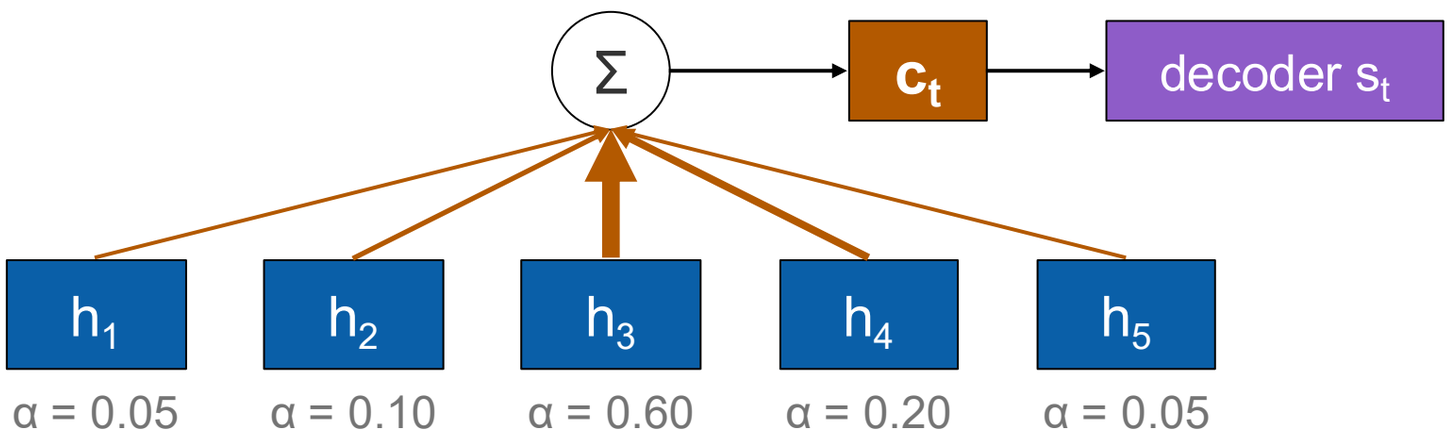
- **Inference**: no ground truth → feed back the model's own predictions (greedy or beam search)
- Train/test mismatch = **exposure bias**: an early mistake puts the decoder in states it never saw
  - Remedy: scheduled sampling (gradually feed own predictions during training)
- Same recipe as next-token training of GPT-style models (5.2)

R. J. Williams, D. Zipser. A learning algorithm for continually running fully recurrent neural networks. Neural Computation, 1989 · S. Bengio et al. Scheduled sampling. NeurIPS 2015

The bottleneck, and the idea of attention

- The whole input is squeezed into **one fixed-size vector** c → quality drops for long sentences
- Bahdanau et al. (2015): keep **all** encoder states $h_1 \dots h_T$; at each output step build a **different context** as a weighted average:

$$c_t = \sum_{i=1}^{T} \alpha_{t,i}\, h_i, \qquad \alpha_{t,i} = \frac{\exp\big(e(s_{t-1}, h_i)\big)}{\sum_j \exp\big(e(s_{t-1}, h_j)\big)}$$



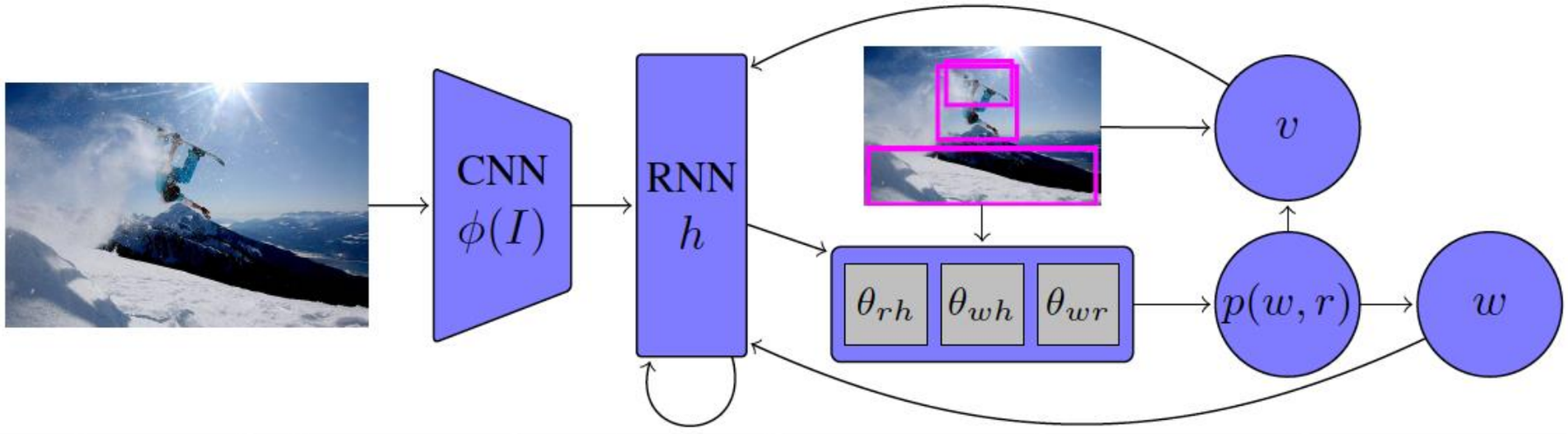
$\alpha_{t,i}$: how much output step t **looks at** input position i (weights sum to 1)

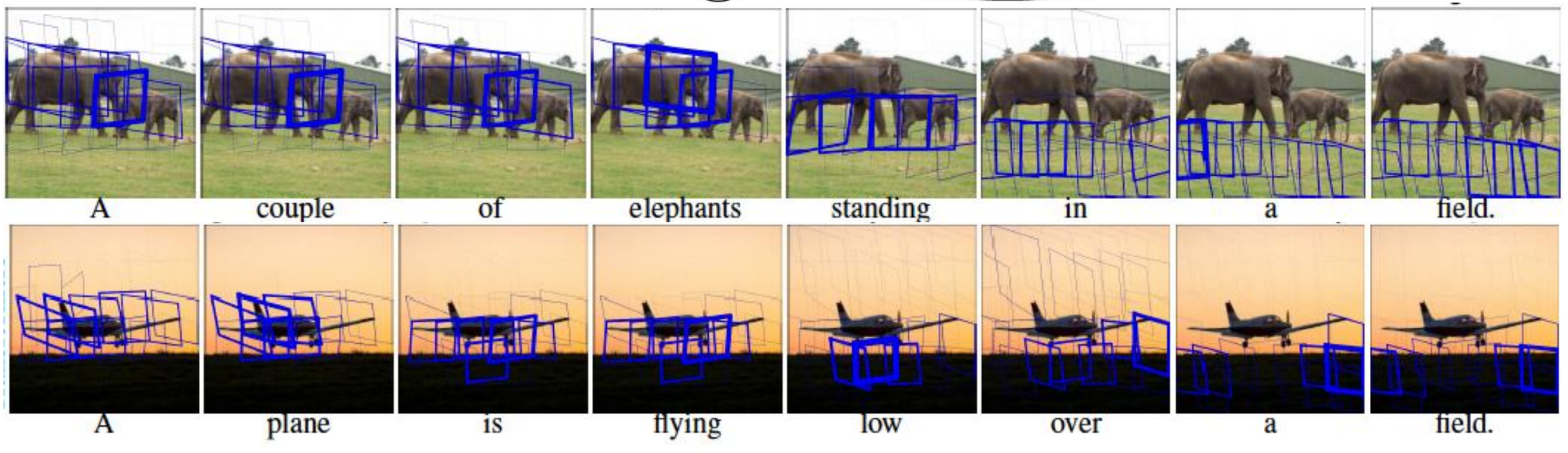
- 5.2: remove the recurrence and keep only attention → **Transformers**

D. Bahdanau, K. Cho, Y. Bengio. Neural machine translation by jointly learning to align and translate. ICLR 2015 · K. Cho et al. On the properties of neural machine translation: encoder–decoder approaches. SSST 2014

RNN applications: captioning with attention

- **Multimodal problems**. Image and Text. Captioning & Attention (other types of sequential data)





Areas of Attention for Image Captioning. Marco Pedersoli, Thomas Lucas, Cordelia Schmid, Jakob Verbeek; ICCV 2017

Summary: key equations

RNN

$$h_t = \tanh\left(W_{hh}h_{t-1} + W_{xh}x_t\right), \quad y_t = W_{hy}h_t$$

BPTT

$$\frac{\partial L}{\partial W_{hh}} \text{ contains } \prod_{j=k+1}^{t} \operatorname{diag}\left(1 - h_j^2\right) W_{hh}, \quad \left\| \prod \cdot \right\| \leq \sigma_{\max}(W_{hh})^{t-k}$$

LSTM

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \quad h_t = o_t \odot \tanh(c_t)$$

Seq2seq

$$p(y \mid x) = \prod_t p\left(y_t \mid y_{<t}, c\right)$$

Attention

$$c_t = \sum_i \alpha_{t,i} h_i, \quad \alpha_{t,.} = \operatorname{softmax}_i\left(e(s_{t-1}, h_i)\right)$$

Main message: recurrence keeps a memory, but long paths through time make learning hard. Gates shorten the gradient path; attention gives direct access to every input.

Homework: char-RNN on your own text

- **Pick your own text** of one style, about the size of the Shakespeare dataset (~200K words)

- Examples: one person's speeches, song lyrics of one genre, source code in one language; more ideas in Karpathy's char-RNN post

- **Run the Colab example** to train a simple RNN on your text: Colab notebook (adapted from the TensorFlow text generation tutorial)

  1. Read and understand the current example

  2. Re-run it with your own data

Resources

- Goodfellow, Bengio, Courville. *Deep Learning*, ch. 10 — www.deeplearningbook.org
- Zhang, Lipton, Li, Smola. *Dive into Deep Learning*, ch. 9–10 — d2l.ai
- A. Karpathy. The unreasonable effectiveness of RNNs — karpathy.github.io (char-RNN code)
- C. Olah. Understanding LSTM networks — colah.github.io
- Stanford cs231n, lecture on RNNs — cs231n.stanford.edu

Slides adapted from A. C. Murillo (Univ. Zaragoza), cs231n (F.-F. Li, J. Johnson, S. Yeung) and MIT 6.869 (B. Freeman, P. Isola)